

Reto 2 – Orquestación de Servicios y Persistencia de Datos

Contexto

Este reto continúa el desarrollo del **sistema de onboarding y offboarding de empleados**. En el reto anterior se construyó un servicio básico para la gestión de empleados y su despliegue en un contenedor Docker.

En este segundo reto se avanzará hacia una arquitectura más realista, incorporando:

- Orquestación de múltiples contenedores con **Docker Compose**
- Persistencia de datos mediante una **base de datos containerizada**
- Comunicación entre **servicios independientes**
- Gestión de **configuración mediante variables de entorno**

Con el segundo servicio aparecen además dos problemas que un sistema de un solo proceso no tiene: **el orden de arranque** —qué pasa cuando un servicio necesita algo que aún no está listo— y **el fallo parcial** —qué hace un servicio cuando su dependencia no responde. Ambos se abordan aquí.

Objetivo

Evolucionar el sistema desarrollado en el reto anterior hacia una arquitectura de múltiples servicios orquestados, donde cada componente se ejecute de forma aislada en su propio contenedor, se comuniquen entre sí mediante la red interna de Docker, y los datos persistan más allá del ciclo de vida de los contenedores.

Al finalizar, el sistema completo debe levantarse desde cero con **un solo comando**, en la máquina de cualquier integrante del equipo, sin pasos manuales.

Requisitos generales

La solución desarrollada debe cumplir con los siguientes requisitos:

- Definir la infraestructura completa en un archivo `docker-compose.yml`.
- Implementar al menos **dos servicios de negocio** que se comuniquen entre sí.
- Incluir una **base de datos** como servicio independiente para cada microservicio.
- Garantizar la **persistencia de datos** mediante volúmenes Docker.
- Definir **cómo se crea el esquema** de cada base de datos, de forma reproducible.
- Configurar **health checks** en el Compose, para que un servicio no arranque antes de que su dependencia esté lista.
- Utilizar **variables de entorno** para la configuración de los servicios.
- Documentar los endpoints de ambos servicios utilizando **OpenAPI (Swagger)**.
- Permitir el despliegue completo mediante un único comando.

Diversidad tecnológica: aquí empieza. El proyecto final exige **al menos 4 lenguajes de programación distintos**, y este reto crea el segundo servicio del ecosistema. Implementelo en un **lenguaje diferente** al del servicio de empleados.

No es un capricho: es lo que hace real el resto del curso. En el Reto 8 tendrá que instrumentar métricas y trazas en cada stack, y en el Reto 7 verá por qué se usa SonarQube en lugar de una herramienta de análisis por lenguaje. Con un solo lenguaje, esas lecciones se quedan en teoría.

Empezar ahora también evita el peor escenario: llegar a la semana 12 con todo en un solo stack y tener que reescribir dos servicios para cumplir la restricción del proyecto.

1. Docker Compose

Crear un archivo `docker-compose.yml`, en este archivo se debe definir toda la infraestructura:

Servicios a incluir

```
services:
  empleados-service:
    # Servicio de gestión de empleados

  departamentos-service:
    # Servicio de gestión de departamentos

  database-empleados:
    # Base de datos (PostgreSQL, MySQL, MongoDB, etc.)

  database-departamentos:
    # Base de datos (PostgreSQL, MySQL, MongoDB, etc.)
```

Inicialmente empiece por registrar en su docker compose el servicio de empleados desarrollado en el reto anterior.

Requisitos del compose

Aspecto	Requisito
Redes	Definir una red interna para la comunicación entre servicios
Volúmenes	Configurar volumen para persistencia de la base de datos
Variables de entorno	Configurar credenciales y URLs de conexión
Dependencias	Usar <code>depends_on</code> para ordenar el inicio de servicios
Puertos	Exponer solo los puertos necesarios al host

Comando de despliegue

La solución debe poder desplegarse con:

```
docker compose up --build
```

Y detenerse con:

```
docker compose down
```

2. Arranque ordenado: `depends_on` no es suficiente

La primera vez que levante el sistema es muy probable que el servicio de empleados falle al arrancar. La causa siempre es la misma: **intentó conectarse a una base de datos que todavía se estaba iniciando**.

`depends_on` no lo evita:

```
empleados-service:
  depends_on:
    - database-empleados    # ⚠ solo espera a que el CONTENEDOR arranque
```

Docker arranca el contenedor de PostgreSQL y de inmediato considera cumplida la dependencia.

Pero

PostgreSQL tarda varios segundos más en aceptar conexiones. Entre esos dos instantes, su servicio ya intentó conectarse y falló.

Contenedor arrancado ≠ servicio listo. Esta distinción es la causa del error de arranque más común en Docker Compose, y reaparecerá cada vez que el ecosistema crezca.

2.1 Health checks: enseñarle a Compose qué significa "listo"

Un `healthcheck` es un comando que Docker ejecuta periódicamente dentro del contenedor.

Mientras

falle, el contenedor está `starting`; cuando pasa, queda `healthy`. Combinado con

`depends_on: condition: service_healthy`, Compose **espera de verdad**:

```

services:
  database-empleados:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: ${DB_EMPLEADOS_NAME}
      POSTGRES_USER: ${DB_EMPLEADOS_USER}
      POSTGRES_PASSWORD: ${DB_EMPLEADOS_PASSWORD}
    volumes:
      - vol-empleados:/var/lib/postgresql/data
    healthcheck:
      # pg_isready responde 0 solo cuando PostgreSQL acepta conexiones
      test: ["CMD-SHELL", "pg_isready -U ${DB_EMPLEADOS_USER} -d ${DB_EMPLEADOS_NAME}"]
      interval: 5s
      timeout: 3s
      retries: 10
      start_period: 10s          # margen inicial sin contar fallos
    networks:
      - microservices-network

empleados-service:
  build: ./empleados
  depends_on:
    database-empleados:
      condition: service_healthy  # ← ahora sí espera a que esté LISTA
  networks:
    - microservices-network

```

2.2 Comando de health check según el servicio

Servicio	Comando
PostgreSQL	<code>pg_isready -U <user> -d <db></code>
MySQL / MariaDB	<code>mysqladmin ping -h localhost</code>
MongoDB	<code>mongosh --eval "db.adminCommand('ping')"</code>
Un microservicio propio	<code>curl -f http://localhost:<puerto>/<ruta> — requiere que curl (o wget) esté en la imagen</code>

Sobre el health check de sus propios servicios: en este reto puede apuntar a cualquier endpoint que responda `200` (por ejemplo `GET /departamentos`). En el **Reto 8** se implementa

un endpoint `/health` formal que verifica también las dependencias del servicio, y esos health checks se actualizarán para usarlo.

2.3 Verificar que funciona

```
docker compose up --build
```

En otra terminal: el estado de salud aparece junto al estado del contenedor

```
docker compose ps
```

```
# database-empleados    ...    Up 30 seconds (healthy)
```

```
# empleados-service    ...    Up 18 seconds
```

El servicio de empleados debe aparecer **después** de que su base de datos esté (healthy) .

2.4 Health check y reintentos resuelven cosas distintas

Es tentador pensar que con health checks ya no hacen falta los reintentos del punto 6. No es así:

Mecanismo	Cuándo actúa	Qué protege
Health check (infraestructura)	Solo en el arranque	Que el servicio no inicie antes de que su dependencia exista
Timeout + reintentos (aplicación)	Durante toda la vida del sistema	Que una dependencia que se cae <i>después</i> no tumbe a quien la consume

Un health check no ayuda en nada si departamentos se cae a las tres de la tarde. Por eso hacen falta los dos.

3. Servicio de Departamentos (nuevo servicio)

Implementar un nuevo microservicio encargado de la gestión de departamentos.

Endpoints requeridos

Método	Ruta	Descripción
POST	/departamentos	Registra un nuevo departamento

Método	Ruta	Descripción
GET	/departamentos/{id}	Consulta un departamento por su identificador
GET	/departamentos	Lista todos los departamentos

Estructura del departamento

```
{  
  "id": "string",  
  "nombre": "string",  
  "descripcion": "string"  
}
```

Respuestas esperadas

- **Registro exitoso:** 201 Created con el departamento creado.
- **Consulta exitosa:** 200 OK con la información del departamento.
- **Departamento no existe:** 404 Not Found con mensaje descriptivo.

Contenerización de la aplicación

- Debe crear un dockerfile para el servicio de departamentos.

Base de datos

- Debe crear una base de datos para el servicio de departamentos.
- Debe crear un volumen para la base de datos.
- Debe crear variables de entorno para la configuración de la base de datos.

Consideraciones

- El servicio de departamentos debe ser independiente del servicio de empleados.
- El servicio de departamentos debe tener su propia base de datos.
- El servicio de departamentos debe tener su propio puerto.
- El servicio de departamentos debe tener su propio volumen.
- El servicio de departamentos debe tener su propia configuración.

4. Servicio de Empleados (evolución del Reto 1)

Evolucionar el servicio desarrollado en el Reto 1 para que:

- Persista los datos en la base de datos (en lugar de memoria).
- Consulte información del **Servicio de Departamentos** al registrar un empleado.
- Obtenga la configuración de conexión a la base de datos desde variables de entorno.

Endpoints requeridos

Método	Ruta	Descripción
POST	/empleados	Registra un empleado asociado a un departamento
GET	/empleados/{id}	Consulta un empleado por su identificador
GET	/empleados	Lista todos los empleados registrados

Estructura del empleado

Es el modelo canónico definido en el Reto 1, sin recortes. Al migrar de memoria a base de datos, todos los campos deben persistirse:

```
{
  "id": "E001",
  "nombre": "Juan",
  "apellido": "Pérez",
  "email": "juan.perez@empresa.com",
  "numeroEmpleado": "EMP-2026-001",
  "cargo": "Desarrollador Senior",
  "area": "Tecnología",
  "departamentoId": "IT",
  "fechaIngreso": "2026-02-10",
  "estado": "ACTIVO"
}
```

El campo `estado` sigue siendo siempre `ACTIVO` en este reto. Las transiciones a `EN_VACACIONES` y `RETIRADO` llegan en los Retos 4 y 5, pero **la columna debe existir desde ya**: agregarla después obliga a una migración de esquema sobre datos existentes.

Validaciones requeridas

Al registrar un empleado, el servicio debe verificar, en este orden:

1. **Unicidad de email** — si ya existe, 400 Bad Request con mensaje descriptivo.
2. **Unicidad de numeroEmpleado** — si ya existe, 400 Bad Request con mensaje descriptivo.
3. **Existencia del departamento** — consultar al **Servicio de Departamentos**; si el departamentoId no existe, 400 Bad Request con mensaje descriptivo.
4. Si todo pasa, registrar el empleado y responder 201 Created .

Las dos primeras validaciones ya existían en el Reto 1, resueltas en memoria. Ahora deben resolverse contra la base de datos — y aquí aparece una decisión que no existía antes: ¿basta con consultar antes de insertar, o hay que declarar además una **restricción UNIQUE** en el esquema? Consultar-y-luego-insertar deja una ventana entre ambas operaciones; la restricción en la base de datos es la única garantía real. Documente qué hizo.

La validación del departamento se debe hacer mediante una petición HTTP REST al servicio de departamentos, con el fin de simular la comunicación básica entre microservicios. En futuros retos se implementarán mecanismos más robustos de comunicación.

Base de datos

- Debe crear una base de datos para el servicio de empleados.
- Debe crear un volumen para la base de datos.
- Debe crear una variable de entorno para la configuración de la base de datos.

Consideraciones importantes

- Utilizar el **nombre del servicio** definido en compose como hostname, y el **puerto interno** del servicio destino — no el puerto publicado al host:

```
http://departamentos-service:8081/departamentos/{id}
```

Dentro de la red de Docker no existe localhost:8081 : cada contenedor resuelve a los demás por su nombre de servicio.

5. Creación del esquema de base de datos

Es la primera vez en el curso que hay una base de datos, y aparece una pregunta que el Reto 1 no tenía: **¿quién crea las tablas, y cuándo?**

No basta con que funcione en su máquina. El criterio es: **un compañero clona el repositorio, ejecuta `docker compose up --build` y el sistema queda operativo, sin pasos manuales.**

5.1 Estrategias

Estrategia	Cómo funciona	Cuándo conviene
Script de inicialización	Un <code>init.sql</code> montado en <code>/docker-entrypoint-initdb.d/</code> del contenedor de la BD. Se ejecuta solo la primera vez , cuando el volumen está vacío	Simple y explícito. Ideal para este reto
Auto-DDL del ORM	El framework crea o actualiza las tablas al arrancar (<code>hibernate.ddl-auto</code> , <code>synchronize: true</code> , <code>Base.metadata.create_all()</code>)	Cómodo en desarrollo. Nunca en producción: puede borrar columnas con datos
Herramienta de migraciones	Flyway, Liquibase, Alembic, <code>node-pg-migrate</code> ... Versiona el esquema en archivos numerados	Es la respuesta profesional. Se estudia a fondo en el Reto 32

Ejemplo de la primera estrategia:

```
database-empleados:
  image: postgres:16-alpine
  volumes:
    - vol-empleados:/var/lib/postgresql/data
    - ./empleados/init.sql:/docker-entrypoint-initdb.d/01-init.sql:ro
```

5.2 La trampa del script de inicialización

`/docker-entrypoint-initdb.d/` **se ejecuta únicamente cuando el volumen de datos está vacío.** Si modifica el `init.sql` y hace `docker compose up`, sus cambios **no se aplican**: el volumen ya tiene datos y PostgreSQL omite la inicialización.

Para forzar que vuelva a ejecutarse hay que borrar el volumen:

```
docker compose down -v      # ⚠️ -v elimina los volúmenes: se pierden TODOS los datos
docker compose up --build
```

Esa limitación —no poder evolucionar el esquema sin perder los datos— es exactamente el problema que resuelven las herramientas de migraciones del Reto 32.

Documente en el README qué estrategia eligió y por qué, y qué debe hacer un compañero para levantar el sistema desde cero.

6. Prepare su servicio para el desastre

Al introducir una llamada de red entre dos servicios, aparece una clase de fallo que no existía en el Reto 1: **la dependencia puede no estar ahí**. Y no es un caso hipotético — lo va a ver en su primer `docker compose up`, porque `depends_on` garantiza el **orden de arranque**, no que el servicio de departamentos ya esté **listo para responder**.

Implemente las dos protecciones mínimas:

- **Timeout explícito** en la llamada HTTP a departamentos. Sin él, una dependencia lenta congela su servicio indefinidamente. Un cliente HTTP sin timeout configurado suele esperar para siempre.
- **Reintentos** si el servicio de departamentos aún no responde, con una espera creciente entre intentos (por ejemplo `1s → 2s → 4s`) y un número máximo.

Y decida qué hacer cuando se agotan los reintentos: ¿rechaza el registro del empleado, o lo acepta marcándolo como pendiente de validación? Documente la decisión y su razón en el README.

Lo que aún NO se pide aquí. Con timeouts y reintentos su servicio sobrevive a un departamento lento o caído, pero **sigue intentándolo cada vez**. Si departamentos está caído diez minutos, su servicio gasta diez minutos reintentando llamadas condenadas a fallar. La solución a eso es el **Circuit Breaker**, y llega en el **Reto 3** junto con el API Gateway, que es donde tiene sentido: cuando el sistema ya tiene un borde definido y algo que proteger.

7. Documentación con OpenAPI

Para facilitar el consumo y prueba de los servicios, debe incorporar la especificación **OpenAPI (Swagger)** en cada uno de sus microservicios.

Requisitos

- Configurar la librería adecuada según su lenguaje de programación (ej: Springdoc para Java/Spring, Swagger UI Express para Node.js, FastAPI incluye soporte nativo).
- Exponer la interfaz gráfica de Swagger UI (ej: `http://localhost:8080/swagger-ui.html` o `http://localhost:8080/docs`).
- Documentar cada endpoint con:
 - Descripción clara de su propósito.
 - Códigos de respuesta HTTP posibles (200, 201, 400, 404, 500).
 - Estructura de los objetos de entrada y salida (Schemas).

8. Pruebas del Sistema

Verificar el funcionamiento completo del sistema:

Flujo de prueba sugerido

1. Iniciar todos los servicios: `docker compose up --build`
2. Crear un departamento:

```
curl -X POST http://localhost:8081/departamentos \
-H "Content-Type: application/json" \
-d '{"id": "IT", "nombre": "Tecnología", "descripcion": "Departamento de TI"}'
```

3. Crear un empleado asociado al departamento, con el modelo completo:

```
curl -X POST http://localhost:8080/empleados \
-H "Content-Type: application/json" \
-d '{"id": "E001", "nombre": "Juan", "apellido": "Pérez",
  "email": "juan.perez@empresa.com", "numeroEmpleado": "EMP-2026-001",
  "cargo": "Desarrollador Senior", "area": "Tecnología",
  "departamentoId": "IT", "fechaIngreso": "2026-02-10"}'
```

4. Verificar que el empleado existe y **conserva todos sus campos**, incluido estado: ACTIVO :

```
GET http://localhost:8080/empleados/E001
```

5. Verificar las tres validaciones (las tres deben responder 400):

```
# Email duplicado
```

```
curl -X POST http://localhost:8080/empleados -H "Content-Type: application/json" \
  -d '{"id":"E002","nombre":"Ana","apellido":"Gómez","email":"juan.perez@empresa.co
    "numeroEmpleado":"EMP-2026-002","cargo":"QA","area":"Tecnología",
    "departamentoId":"IT","fechaIngreso":"2026-02-11"}'
```

```
# Número de empleado duplicado
```

```
curl -X POST http://localhost:8080/empleados -H "Content-Type: application/json" \
  -d '{"id":"E003","nombre":"Ana","apellido":"Gómez","email":"ana@empresa.com",
    "numeroEmpleado":"EMP-2026-001","cargo":"QA","area":"Tecnología",
    "departamentoId":"IT","fechaIngreso":"2026-02-11"}'
```

```
# Departamento inexistente
```

```
curl -X POST http://localhost:8080/empleados -H "Content-Type: application/json" \
  -d '{"id":"E004","nombre":"Ana","apellido":"Gómez","email":"ana@empresa.com",
    "numeroEmpleado":"EMP-2026-004","cargo":"QA","area":"Ventas",
    "departamentoId":"N0-EXISTE","fechaIngreso":"2026-02-11"}'
```

Prueba de persistencia: el contraste que importa

Reiniciar con `restart` no prueba nada — el contenedor nunca se destruyó. La prueba real es **destruir los contenedores y verificar que los datos siguen ahí**, y luego contrastarlo con la destrucción del volumen:

```
# 1. Destruir los contenedores, CONSERVANDO los volúmenes
```

```
docker compose down
```

```
docker compose up -d
```

```
curl http://localhost:8080/empleados/E001
```

```
# ✅ El empleado sigue existiendo: los datos viven en el volumen, no en el contenedor
```

```
# 2. Ahora destruir TAMBIÉN los volúmenes
```

```
docker compose down -v
```

```
docker compose up -d --build
```

```
curl http://localhost:8080/empleados/E001
```

```
# ❌ 404: el volumen se borró y con él los datos
```

Esa es la diferencia entre `down` y `down -v`. El segundo comando reaparecerá en el Reto 7, cuando el pipeline de CI necesite un entorno limpio entre ejecuciones de las pruebas E2E — ahí borrar los volúmenes es justamente lo que se busca.

Prueba del arranque ordenado

```
# Levantar desde cero y observar el orden
docker compose down -v
docker compose up --build
```

```
# En los logs, el servicio de empleados NO debe reportar errores de conexión a la BD.
# Si los reporta, el health check no está bien configurado.
```

Decisión técnica requerida

Este reto tiene tres decisiones que no tienen una única respuesta correcta. **Justifíquelas en el README**; se evalúa el argumento, no la opción elegida.

- **Motor de base de datos por servicio.** ¿Usa el mismo motor para ambos, o aprovecha que son servicios independientes para usar motores distintos? La *persistencia poliglota* es una de las ventajas que se le atribuyen a los microservicios; también multiplica lo que el equipo debe saber operar. ¿Qué ganó y qué costó su elección?
- **Creación del esquema** (§5). ¿Script de inicialización, auto-DDL del ORM o migraciones? ¿Qué pasa cuando el esquema tenga que cambiar y ya haya datos?
- **Garantía de unicidad** (§4). ¿Consulta previa, restricción `UNIQUE` en el esquema, o ambas? ¿Qué ocurre si dos peticiones con el mismo email llegan al mismo tiempo?

Entregables

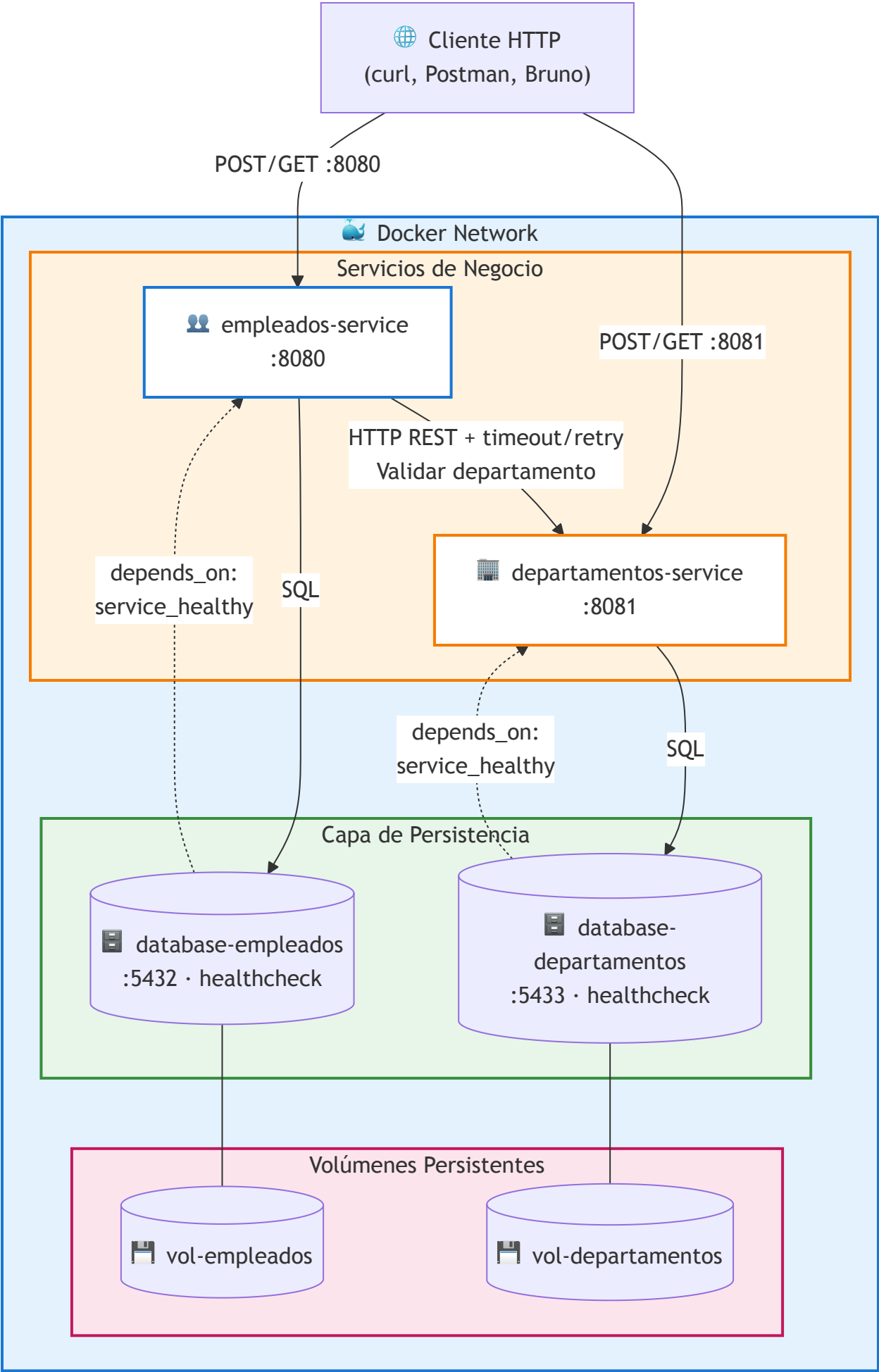
- Código versionado en GitHub, con un `Dockerfile` por servicio.
- `docker-compose.yml` con redes, volúmenes, variables de entorno y **health checks** configurados.
- Script o mecanismo de **creación del esquema** de cada base de datos.
- [README.md](#) del sistema completo que incluya:

- Instrucciones para levantar todo desde cero, **verificables por alguien que clona el repositorio**.
- Tabla **servicio** ↔ **lenguaje** ↔ **motor de base de datos** ↔ **puerto**.
- Las tres decisiones técnicas justificadas.
- Evidencia de la **prueba de persistencia**: el contraste entre `down` y `down -v` .
- Evidencia del **arranque ordenado**: salida de `docker compose ps` mostrando la BD (`healthy`) .
- Cómo acceder a la Swagger UI de cada servicio.
- [README.md](#) por servicio con sus endpoints y su configuración propia.

Consideraciones

- Cada servicio debe tener su propio `Dockerfile` , su propia base de datos y su propia configuración.
- **Ningún servicio accede a la base de datos de otro**. Si empleados necesita un dato de departamentos, lo pide por HTTP. Compartir la base de datos entre servicios es el antipatrón que convierte un sistema de microservicios en un monolito distribuido.
- Las credenciales de base de datos van en variables de entorno, no en el código. En el **Reto 9** se refuerza sacándolas también del `docker-compose.yml` .
- Los endpoints `PUT` y `DELETE` del CRUD completo llegan en el **Reto 4**, junto con los eventos que publican. Aquí basta con `POST` y `GET` .

Diagrama de Arquitectura Esperada



Criterios de Evaluación

El reto se evalúa sobre **5 puntos**, distribuidos de la siguiente forma:

#	Elemento	Valor	Aspectos a evaluar
1	Docker Compose y arranque ordenado	1.0	<code>docker-compose.yml</code> con redes, volúmenes, variables de entorno y puertos correctos. Health checks configurados en las bases de datos y <code>depends_on: condition: service_healthy</code> en los servicios que dependen de ellas. El sistema levanta desde cero sin errores de conexión.
2	Servicio de Departamentos	1.0	Endpoints completos, Dockerfile, base de datos propia con su volumen, configuración por variables de entorno y documentación OpenAPI . Implementado en un lenguaje distinto al de empleados.
3	Servicio de Empleados	1.0	Migración de memoria a base de datos conservando el modelo canónico completo del Reto 1 (los 10 campos, incluido <code>estado</code>). Las tres validaciones funcionando: unicidad de <code>email</code> , unicidad de <code>numeroEmpleado</code> y existencia del departamento. Comunicación REST con departamentos. Documentación OpenAPI .
4	Persistencia y esquema	1.0	Mecanismo de creación del esquema reproducible y documentado. Evidencia del contraste <code>down</code> (los datos sobreviven) vs. <code>down -v</code> (se pierden). Cada servicio con su propia base de datos, sin accesos cruzados.
5	Tolerancia a fallos y pruebas	1.0	Timeout explícito y reintentos con espera creciente en la llamada a departamentos, con comportamiento definido al agotarlos. Evidencia de las pruebas del punto 8, incluidas las tres validaciones rechazando con <code>400</code> . Swagger UI funcionando en ambos servicios.

Nota: La documentación ([README.md](#)) y el versionamiento adecuado en GitHub son requisitos transversales que afectan la evaluación de cada elemento. Las **tres decisiones técnicas** deben

estar justificadas; una respuesta ausente o sin argumento descuenta del criterio correspondiente.